Astana IT University · Computer Vision · Assignment 2

Hand Gesture Recognition: making the A1 model work in real conditions

Tokhtamishov Bekhruz · SE-2422 · Instructor: Nurym Kaiyrkhan · November 2026

Summary

In Assignment 1 my models got 100% on the test set. But the test frames came from the same videos as the training frames, so this number was too optimistic. In Assignment 2 I did four things:

1. I made a harder test: the same test photos with dark light, blur, noise, a new background and short moving clips.

2. I added modern (SOTA) models: DINOv2, SAM 2.1 and Google's MediaPipe gesture model.

3. I explained the answers with pictures: hand points, attention maps, hand masks, similar images.

4. I built a small website with a consent page, live camera, latency and browser information.

Main result: the A1 CNN falls from 100% to 62.3% under these conditions. The new model keeps 96.9%.

Step 1. Data

I use the same data as in A1: 450 frames from three iPhone videos of my hand, 150 per gesture.

Class	Train	Validation	Test
thumbs_up	105	22	23
peace	105	22	23
stop	105	22	23
total	**315**	**66**	**69**

When I looked at the data again, I found three problems:

- **Wrong labels:** around 10 thumbs_up frames show a half-open hand (from the video between two gestures).
- **Background hints:** thumbs_up frames often show my black sleeve; stop frames have a dark wall. A model can learn these instead of the hand.
- **Near copies:** test frames are almost the same as some train frames (similarity 0.93–0.94). This is why A1 showed 100%.

Step 2. A harder test (synthetic conditions)

I made 7 new versions of every test photo. Together with the clean photo this gives 8 × 69 = 552 test images; 483 of them (87.5%) are synthetic. All synthetic files start with SYN_ and have a red SYNTHETIC label in the app.



Figure 1. One test photo under the 8 test conditions.

For the new background I cut out the hand with SAM 2.1 (Step 3). Training uses other background styles than the test, so the model never sees the test backgrounds. During training 70% of images get a random condition.

Where in the code: `src/stress.py, src/make_stress_set.py`

Step 3. SOTA models and course ideas

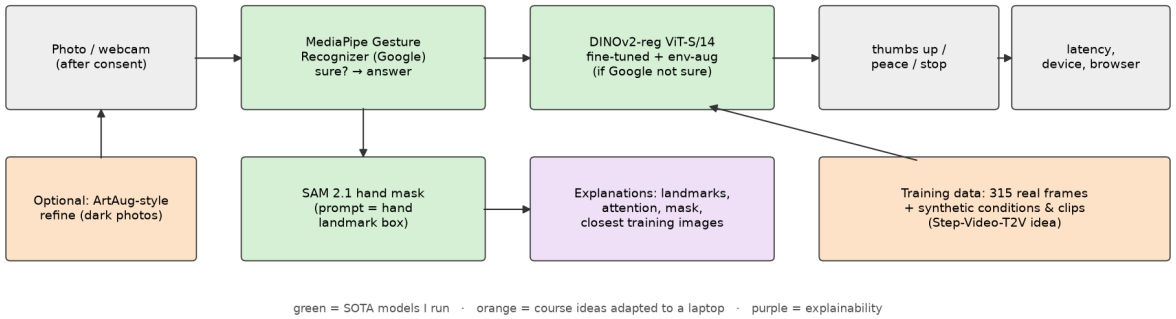


Figure 2. How the system works.

Component	What it does	Why
DINOv2-reg ViT-S/14 (Meta, 2024)	Main image classifier. I fine-tune the last 4 of 12 blocks.	Strong features from 142M images; learns from only 315 photos.
SAM 2.1 (Meta, 2024)	Cuts the hand out of the photo (mask).	Needed for the background test and for the explanation. Base model of MedSAM2 from the course list [3].
MediaPipe Gesture Recognizer (Google)	Pretrained gesture model with Thumb_Up, Victory, Open_Palm.	Trained on many different hands; works with a webcam, a face in the frame and new angles.
MediaPipe hand landmarks + my MLP	21 hand points → small network trained on my data.	Used for hand points, the SAM prompt and comparison.
Step-Video-T2V idea [1]	Short synthetic clips with a moving camera.	The real model needs ~80 GB GPU memory; I make the clips from my photos.
ArtAug idea [2]	Check the photo (too dark? noisy?) and fix it.	The real model uses FLUX + Qwen2-VL-72B; mine is a small version.

Where in the code: `src/models.py, src/landmarks.py, src/stress.py (SAM 2.1), src/clips.py, src/enhance.py`

Step 4. Training

Model	Input	Optimizer	Parameters	Time (M2)
A1 CNN	4 conv blocks, 128 px	Adam 1e-3, 15 epochs	0.50M	21 s
DINOv2-reg + env-aug	ViT-S/14, 224 px, 4 blocks trained	AdamW 2e-5 / 1e-3, 10 epochs	7.1M of 21.6M	126 s
Landmark MLP	78 features, 243 frames ×11 aug	Adam 3e-3, 80 epochs	9k	< 10 s

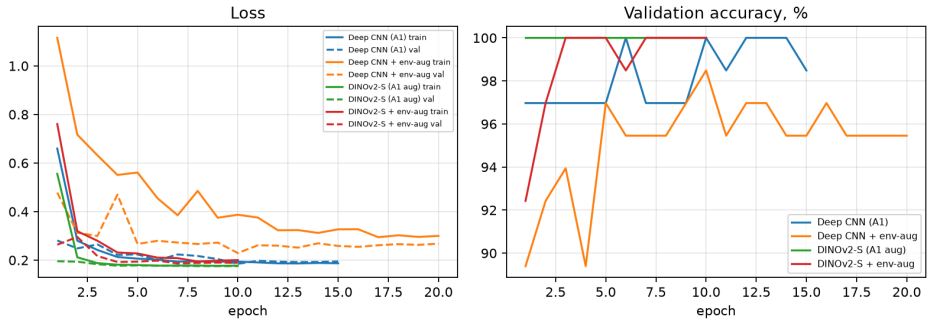


Figure 3. Loss and validation accuracy. Validation is clean, so it is close to 100% quickly.

Where in the code: `src/train.py, src/train_landmarks.py`

Step 5. Results

Table 1. Accuracy (%) on 69 test photos per condition. Last two columns: average over the 7 synthetic conditions.

Model	clean	dark	bright	blur	noise	rotate	new bg	night	avg acc	avg F1
LogReg (A1)	100.0	100.0	91.3	100.0	100.0	65.2	26.1	100.0	83.2	81.1
CNN (A1)	100.0	33.3	36.2	98.6	100.0	85.5	49.3	33.3	62.3	55.4
DINOv2-reg	100.0	100.0	100.0	78.3	98.6	89.9	46.4	59.4	81.8	80.4
DINOv2-reg + env-aug	**100.0**	**100.0**	**98.6**	**97.1**	**100.0**	**100.0**	**100.0**	**82.6**	**96.9**	**96.9**
Auto: Google → DINOv2	100.0	100.0	98.6	97.1	100.0	94.2	98.6	82.6	95.9	95.9

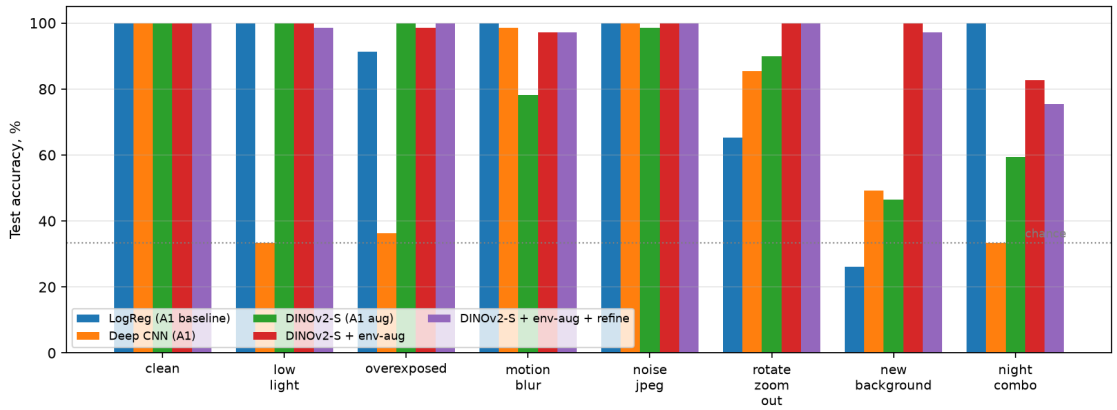


Figure 4. Accuracy per condition. Dotted line = random guess (33%).

- **Clean:** all models get 100% on clean photos. So the A1 test could not show the difference.
- **Dark:** the A1 CNN says "thumbs up" for every dark photo (33.3%). DINOv2 gets 100%.
- **New background:** LogReg drops to 26.1%: it was looking at the background, not the hand.
- **Auto:** Auto (95.9%) is a little lower than DINOv2 alone on synthetic photos, but it works much better with a real webcam (Step 8).

Ablation: which part helps?

Variant	avg acc	dark	new bg	night
CNN (A1)	62.3	33.3	49.3	33.3
CNN + env-aug	93.6	85.5	95.7	85.5
DINOv2-reg, no env-aug	81.8	100.0	46.4	59.4
DINOv2-reg + env-aug, no background swap	90.1	100.0	47.8	84.1
DINOv2-reg + env-aug (final)	**96.9**	**100.0**	**100.0**	**82.6**
final + ArtAug-style refine	95.4	98.6	97.1	75.4

- **Pretrained backbone** fixes dark and bright photos (CNN 33% → DINOv2 100% in the dark).
- **Synthetic augmentation** also helps the small CNN (62.3% → 93.6%).
- **Background swap** is the most important part: 100.0% with it, 47.8% without it.
- **ArtAug-style refine** did not help (82.6% → 75.4% at night): the model already learned dark photos, and brightening makes JPEG noise stronger. It is an option in the app, not default.

Where in the code: `src/evaluate.py, src/eval_cascade.py → results/metrics.json, results/cascade_metrics.json`

Step 6. Synthetic clips (Step-Video-T2V idea)

From every test photo I made 4 clips of 2 seconds (16 frames): a shaking camera, a moving camera in a new room, a dark shaking camera, and a fast camera at night. Every frame is classified.

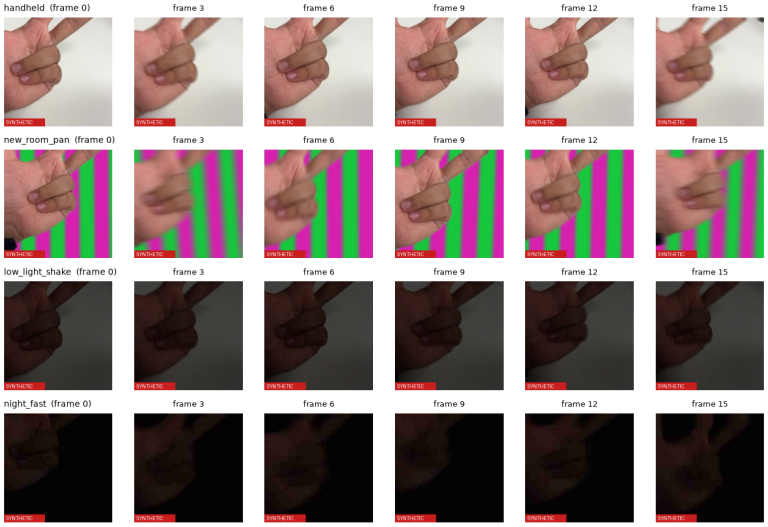


Figure 5. Frames from the four clip types (synthetic).

Model (% frames correct)	handheld	new_room_pan	low_light_shake	night_fast
CNN (A1)	100.0	48.2	33.3	33.3
DINOv2-reg	100.0	44.6	98.3	43.7
DINOv2-reg + env-aug	**99.9**	**94.4**	**99.7**	**58.2**

The fast night clip is still hard: 58.2% of frames are correct.

Where in the code: `src/clips.py → results/clips/*.mp4, results/clips_metrics.json`

Step 7. Explainability

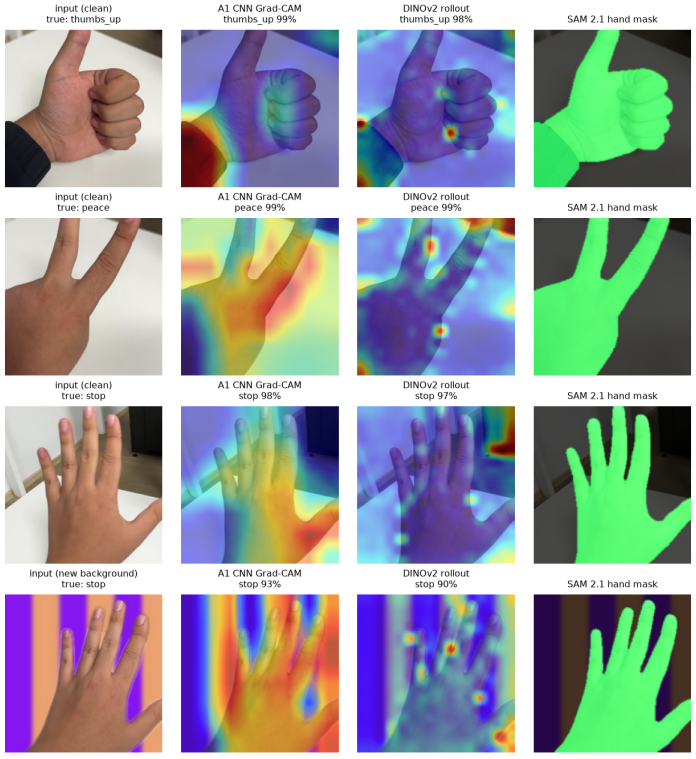


Figure 6. Grad-CAM (A1 CNN), attention rollout (DINOv2) and SAM 2.1 hand mask.

- **Top row:** the A1 CNN says “thumbs up 99%”, but its heatmap is on the black sleeve, not on the thumb. This explains why it says “thumbs up” for all dark photos.

- **Mask:** the SAM 2.1 mask shows where the hand really is. In the app SAM gets its box from the MediaPipe hand points, so it never selects the face.
- **Similar images:** the app also shows the most similar training images. They show what the model “remembers”.

Where in the code: `src/explain.py (Grad-CAM, attention rollout), src/landmarks.py`

Step 8. The web app

The app is my own website (FastAPI + HTML/JS), not a template. It runs on my laptop at http://127.0.0.1:7860 and has five pages.

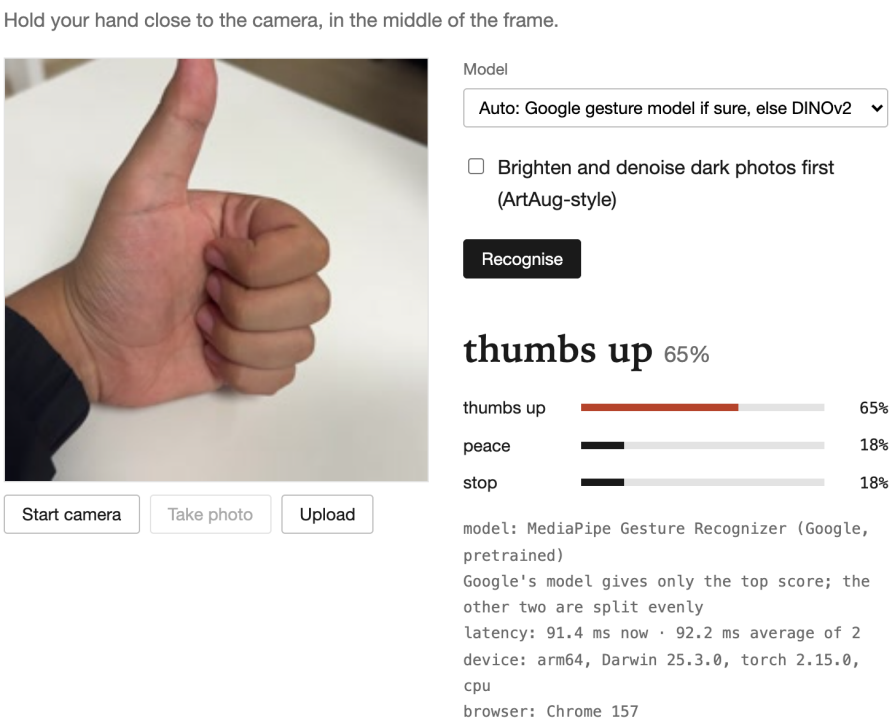


Figure 7. Recognise page: photo, answer, latency (now and average), device and browser. Below it the page shows hand points, attention, the SAM 2.1 mask and similar images.

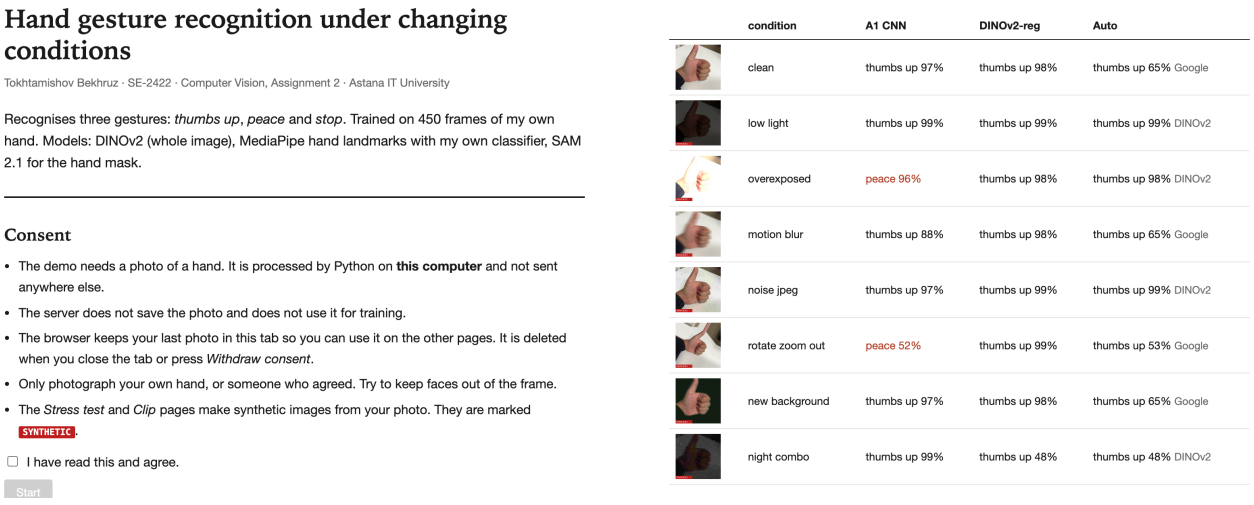


Figure 8. Left: Start page with the consent form; the other pages do not work before the user agrees. Right: Stress test page, red = the answer changed.

Page	What you see
Start	Consent text and checkbox. Without consent the other pages send you back.

Recognise	Webcam or upload → answer → hand points, attention, SAM mask, similar images, latency (now and average), device, browser.
Stress test	Your photo under 8 conditions; A1 CNN vs DINOv2 vs Auto.
Clip	A 2-second synthetic clip from your photo; answer for every frame.
Results	Tables and charts from this report.

Model (one photo, MacBook M2 CPU)	Latency
A1 CNN	2.0 ms
DINOv2-reg	24.2 ms
MediaPipe Gesture Recognizer	about 30–45 ms (measured in the app)

Real webcam test: with my face in the frame and the thumb seen from the front, DINOv2 alone said “peace”. MediaPipe's gesture model was right on all my webcam photos, because it was trained on many different hands. This is why Auto asks Google's model first.

Where in the code: `app.py (API), web/*.html, web/common.js, web/style.css`

Step 9. Ethics and limits

- **Consent:** the user must agree before the camera or upload works. Photos are processed in memory and not saved on the server.
- **Synthetic media:** every synthetic image and clip has a red SYNTHETIC label and the SYN_ file prefix.
- **Bias:** only my hand, one skin tone, two rooms. The models may work worse for other people.
- **Limits:** the stress test is synthetic and made by me, so it is easier than real new photos. A real test with new photos in other rooms is the next step (the code supports a data/real_new_place/ folder).
- **Misuse:** hand cut-out and background swap could be misused to fake where a photo was taken. Here it only works on the user's own photo.

Step 10. How to run

```
python3.12 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python src/make_stress_set.py && .venv/bin/python src/train.py
.venv/bin/python src/train_landmarks.py && .venv/bin/python src/evaluate.py
.venv/bin/python src/extras_light.py && .venv/bin/python src/eval_cascade.py && .venv/bin/python src/clips.py
.venv/bin/python app.py        # open http://127.0.0.1:7860
```

Python 3.12, PyTorch 2.15, timm 1.0.31, transformers 5.22, MediaPipe, FastAPI. Exact versions: requirements.txt. The A1 dataset must be in ../dataset. Seed 42.

References

[1] Neurohive. Step-Video-T2V: Text-to-Video Open-Source Model Achieves 16x Video Compression Breakthrough.

[2] Neurohive. ArtAug: Open Source Framework for Image Generation Models Enhancement.

[3] Neurohive. MedSAM2: Open Source SOTA 3D Medical Image and Video Segmentation Model.

[4] Oquab et al. DINOv2: Learning Robust Visual Features without Supervision. 2024. Darcet et al. Vision Transformers Need Registers. ICLR 2024.

[5] Ravi et al. SAM 2: Segment Anything in Images and Videos. ICLR 2025.

[6] Google. MediaPipe Gesture Recognizer and Hand Landmarker. ai.google.dev/edge/mediapipe.

[7] Selvaraju et al. Grad-CAM. ICCV 2017. Abnar & Zuidema. Attention rollout. ACL 2020.